UNIVERSITY OF PUNJAB ALLAMA IQBAL CAMPUS

Roll No : BCSF24M001

Name : Muhammad Huzaifa Farrukh
Section : BCSF24 Morning
Session : 2024-28
Course Name : Computer Networks
Instructor Name : Dr.Madeha Aman

Date : 03/05/26

1. Project Title

Network Traffic Monitoring and Analysis Platform — "**NetWatch v1.0".**

2. Abstract

NetWatch is a real-time network traffic monitoring and analysis platform built using C++ for the backend packet-capture engine and HTML, CSS, and JavaScript for the front-end dashboard. The system uses the Npcap (libpcap-compatible) library to perform live packet sniffing on a selected Wi-Fi or Ethernet interface, parses Ethernet → IPv4 → TCP/UDP/ICMP headers, maps well-known port numbers to their corresponding application services, and streams the captured records to a browser-based dashboard via a lightweight HTTP API. The user can start and stop monitoring on demand, filter records by protocol, source IP, or destination IP, and view live statistical summaries including total packets, per-protocol counts, and average packet size.

3. Introduction

Modern computer networks generate enormous volumes of traffic every second. Understanding what is flowing through a network — which protocols dominate, which hosts communicate, and what services are being used — is fundamental to network administration, performance tuning, and cyber-security. This project applies the core concepts of the Computer Networks course (the OSI/TCP-IP layered model, IP addressing, transport-layer protocols, and well-known ports) to build a working tool that captures real packets directly off the wire and presents them in a human-readable form.

Unlike a simulated dataset, NetWatch performs true real-time packet sniffing using the Npcap driver on Windows, which qualifies the project for the bonus criterion mentioned in the project specification.

4. Objectives

- Capture live network packets from a chosen network interface.
- Decode Ethernet, IPv4, TCP, UDP, and ICMP headers correctly.
- Map common destination ports to their application-layer services (HTTP, HTTPS, DNS, SSH, etc.).
- Provide Start / Stop monitoring controls through a clean web interface.
- Display per-packet records and live statistics (total, per-protocol, average size).
- Allow filtering of records by protocol, source IP, and destination IP.

5. Tools and Technologies Used

Component	Technology
Backend Engine	C++17 with Npcap SDK 1.16 (libpcap-compatible)
HTTP Server	Custom lightweight Winsock2 HTTP server (port 8080)
Frontend	HTML5, CSS3, vanilla JavaScript (ES6)
Capture Driver	Npcap (Windows packet capture library)
Build Tool	g++ (MinGW-w64) with C++17
Platform	Windows 10 / 11 (64-bit)

Compilation command:

```
g++ server.cpp -o server.exe -I"C:/npcap-sdk-1.16/Include" -L"C:/npcap-sdk-1.16/Lib/x64" -
lwpcap -lPacket -lws2_32 -ladvapi32 -lpthread -std=c++17 -O2 -D_WIN32_WINNT=0x0601
```

6. System Architecture

NetWatch follows a clean two-tier architecture in which the C++ backend performs all heavy network operations and the browser-based frontend handles only presentation and interaction.

6.1 Layered Flow

- Layer 1 – Capture: Npcap delivers raw Ethernet frames from the NIC to server.exe.
- Layer 2 – Parsing: server.cpp decodes Ethernet → IPv4 → TCP/UDP/ICMP headers and stores PacketRecord objects.
- Layer 3 – API: An embedded HTTP server exposes /api/devices, /api/start, /api/stop, /api/packets, and /api/stats.
- Layer 4 – Presentation: index.html + app.js poll the API every ~1.5 seconds and update the dashboard.

7. Implementation Details

7.1 Backend (server.cpp)

The backend is implemented in modern C++17. Key elements include:

- Packed structs for EthernetHeader, IPv4Header, TCPHeader, UDPHeader, and ICMPHeader to map raw bytes safely.
- A pcap_loop callback that filters packets using the BPF expression "ip" so only IPv4 traffic enters the analyser.
- A protocol dispatcher that recognises TCP (6), UDP (17), and ICMP (1); all other protocols are tagged as OTHER.

- A port-to-service map that converts numeric ports to readable service names (HTTP, HTTPS, DNS, SSH, FTP, SMTP, etc.).
- Thread-safe shared state protected by std::mutex and std::atomic<bool> for clean Start/Stop control.
- A minimal Winsock2 HTTP server that emits JSON responses consumed by the frontend.

7.2 Frontend (index.html, style.css, app.js)

- Clean cyber-themed dashboard with a fixed header, control bar, statistics grid, filter bar, and packets table.
- Device dropdown populated from /api/devices on page load.
- Start, Stop, and Clear buttons that call the corresponding API endpoints.
- Polling loop (setInterval ~1500 ms) that fetches new packets and statistics and re-renders incrementally.
- Client-side filters for Protocol, Source IP, and Destination IP, plus a Reset button.

8. Functional Requirements Coverage

Requirement	Status	Implemented In
Read / capture network traffic	✓	server.cpp (pcap)
Display Source IP	✓	Packets Table
Display Destination IP	✓	Packets Table
Identify Protocol (TCP/UDP/ICMP)	✓	server.cpp parser
Map ports to services	✓	portToService()
Count packets	✓	Stats grid
Maintain logs	✓	Log Console
Start / Stop control	✓	API + buttons
Statistics (total, per-proto, avg size)	✓	/api/stats
Filter by Protocol / Src IP / Dst IP	✓	app.js filters
Real-time packet sniffing (BONUS)	✓	Npcap live capture

9. Dataset Fields Captured

Each PacketRecord generated by NetWatch contains the following fields, matching the dataset specification in the project brief:

Field	Example Value
Time	10:35:21
Source IP	192.168.100.17
Destination IP	8.8.8.8
Protocol	TCP
Source Port	52341
Destination Port	443
Service	HTTPS
Packet Size	512 bytes

10. How to Run

- Install Npcap on Windows (with the "WinPcap-compatible mode" option).
- Compile server.cpp using the g++ command shown in Section 5.
- Run server.exe as Administrator (required for raw packet capture).
- Open index.html in any modern browser.
- Choose the active Wi-Fi interface (Intel® Wi-Fi 6 AX203) from the dropdown and click START.
- Browse any website to observe TCP, UDP, and ICMP packets streaming live into the table.

11. Verification of Real-Time Capture

To prove that the capture is genuinely real-time and not a simulation, the following tests are performed during the demonstration:

- ICMP Test: running ping 8.8.8.8 from the command prompt produces exactly four ICMP rows in the dashboard with matching timestamps.
- Airplane-mode Test: toggling Wi-Fi off freezes the packet stream within ~1 second; toggling it back on resumes it instantly.
- HTTP Test: visiting http://neverssl.com produces visible TCP packets on port 80 (service = HTTP).
- Cross-validation with Wireshark: running Wireshark in parallel shows nearly identical packet counts and timing.

12. Conclusion

NetWatch successfully demonstrates the foundational concepts of network traffic monitoring covered in the Computer Networks course. The project goes beyond the minimum requirements by performing real-time packet sniffing using Npcap, qualifying for the bonus criterion. Through this project, practical understanding was gained of how the OSI/TCP-IP layers cooperate, how IP and transport-layer headers

are structured, how libpcap-style capture drivers work, and how a backend service can communicate with a browser front-end through a simple REST-style HTTP API.

13. References

- Npcap Project — https://npcap.com
- libpcap Documentation — https://www.tcpdump.org
- Andrew S. Tanenbaum, “Computer Networks”, 5th Edition.
- IANA Service Name and Transport Protocol Port Number Registry.